

Welcome back! Ab tak humne LLM ko instructions dena (Prompts), output clean karna (Parsers), aur memory dena seekha. Lekin ek major problem abhi bhi hai: **LLMs ke paas aapka private data nahi hai**. Agar aap apne "HealthBuddy" ecosystem mein AI se kisi specific medical device ke manual ya patient logs ke basis par sawal poochna chahte hain, toh ChatGPT directly answer nahi de payega. Hamein wo documents LLM tak pohochane honge. Yahin par LangChain ke **Document Loaders** data ingestion pipeline ka pehla step bante hain. Let's master how to bring external data into the AI world.

Module 1: The Core Concept of Documents

1. Why document loading is needed

Real-world applications (jaise Retrieval-Augmented Generation ya RAG) tabhi useful hoti hain jab AI aapke custom data par answer kare. Data alag-alag formats (PDF, CSV, Notion, SQL) mein hota hai. In sab formats ko padh kar ek aisi standard language mein convert karna jo LLM samajh sake, **Document Loaders** ka kaam hai. Yeh ek ETL (Extract, Transform, Load) pipeline ke "Extract" phase jaisa hai.

2. What is a Document object? & 3. Metadata and Page Content

LangChain mein jab bhi koi loader data padhta hai, toh wo us data ko ek standard Python object mein wrap kar deta hai jise **Document** kehte hain.

- **Real-world Analogy:** Socho aapke paas ek post office ka Envelope hai.
 - **page_content (The Letter):** Yeh envelope ke andar ka actual text/string hai (e.g., PDF ka actual text).
 - **metadata (The Address Label):** Yeh ek dictionary hai jo batati hai ki yeh text kahan se aaya (e.g., {"source": "manual.pdf", "page": 4, "author": "Dr. Smith"}). Metadata aage chalkar data filter karne mein bohot kaam aata hai.

Module 2: The Loaders Deep Dive

LangChain ke paas 100+ loaders hain, par industry mein yeh 3 sabse zyada use hote hain.

A. TextLoader (.txt, .md, logs)

- **How it works internally:** Yeh Python ke standard `open(file).read()` function ka wrapper hai. Yeh poori file ko uthata hai aur ek single Document object bana deta hai.
- **When to use it:** Server logs, raw code files, markdown notes, ya simple text dumps read karne ke liye.
- **Real-world use case:** Reading server error logs to ask the LLM, "Summarize the critical errors from yesterday."

Complete Python Code & Execution:

```

from langchain_community.document_loaders import TextLoader

# 1. Initialize Loader
# Note: Always specify encoding to prevent Windows/Linux UTF-8 crashes
loader = TextLoader("server_logs.txt", encoding="utf-8")

# 2. Load the file
docs = loader.load()

# 3. Inspecting Document and Metadata
print(f"Total documents loaded: {len(docs)}") # Output: 1 (Entire file is 1 doc)

first_doc = docs[0]
print(f"Metadata: {first_doc.metadata}")
# Output: {'source': 'server_logs.txt'}

print(f"Content snippet: {first_doc.page_content[:100]}...")
# Output: "ERROR 2026-06-28: Database connection timeout..."

```

- **Common Error:** UnicodeDecodeError. Hamesha encoding="utf-8" pass karein, khaaskar agar text file mein special characters hain.

B. CSVLoader (.csv)

- **How it works internally:** Yeh CSV ko row-by-row parse karta hai. **Har ek row ek naya Document ban jati hai.** Yeh columns ke headers ko as 'keys' aur cell data ko as 'values' format karke page_content mein dalta hai.
- **When to use it:** Tabular data, database exports, ya excel sheets (converted to CSV).
- **Real-world use case:** Processing a CSV of patient triage logs to find patterns in medical device usage.

Complete Python Code & Execution:

```

from langchain_community.document_loaders import CSVLoader

# Assume patient_records.csv has columns: PatientID, Symptoms, DeviceAssigned

# 1. Initialize Loader

```

```
loader = CSVLoader(file_path="patient_records.csv")
```

```
# 2. Load Data
```

```
docs = loader.load()
```

```
# 3. Inspecting the output
```

```
print(f"Total rows (documents) loaded: {len(docs)}")
```

```
# Output: e.g., 500 (If CSV has 500 rows)
```

```
print(f"First Row Metadata: {docs[0].metadata}")
```

```
# Output: {'source': 'patient_records.csv', 'row': 0}
```

```
print(f"First Row Content:\n{docs[0].page_content}")
```

```
# Output:
```

```
# PatientID: P-101
```

```
# Symptoms: Severe lower back pain
```

```
# DeviceAssigned: CT Scan
```

- **Common Error:** Giant CSV files memory crash kar sakti hain. Aise case mein hum load() ki jagah .lazy_load() use karte hain jo iterator return karta hai (memory save karne ke liye).

C. PDFLoader (using PyPDF)

- **How it works internally:** Yeh binary PDF file ko decode karta hai. Har page ko read karke ek separate Document banata hai. So a 10-page PDF returns a list of 10 Documents.
- **When to use it:** Books, research papers, device manuals, or invoices.
- **Real-world use case:** Building a "Chat with PDF" app for M.Tech students to quickly query large AI/ML research papers.

Complete Python Code & Execution:

```
# pip install pypdf
```

```
from langchain_community.document_loaders import PyPDFLoader
```

```
# 1. Initialize
```

```
loader = PyPDFLoader("medical_imaging_handbook.pdf")
```

```
# 2. Load (Extracts page by page)
```

```
docs = loader.load()
```

```
# 3. Inspect
print(f"Total pages loaded: {len(docs)}") # Output: 45

page_5 = docs[4] # 0-indexed
print(f"Page 5 Metadata: {page_5.metadata}")
# Output: {'source': 'medical_imaging_handbook.pdf', 'page': 4}

print(f"Page 5 Content snippet: {page_5.page_content[:200]}")
```

- **Common Error: Scanned PDFs.** PyPDF sirf digital text read kar sakta hai. Agar PDF mein sirf scanned images hain, toh page_content empty aayega. Uske liye humein OCR loaders (like UnstructuredPDFLoader with Tesseract) use karna padta hai.

Module 3: Comparison & Practice

Loader Comparison Table

Feature	TextLoader	CSVLoader	PyPDFLoader
Input Format	.txt, .md, .log	.csv	.pdf
Document Splitting	1 File = 1 Document	1 Row = 1 Document	1 Page = 1 Document
Metadata Details	Source file path	Source file + Row number	Source file + Page number
Pros	Super fast, 100% accurate	Auto-formats tabular data well	Perfect for unstructured reports
Cons	Fails with complex encodings	Bad for very large ML datasets	Fails on image-only scanned PDFs

Practice Exercises

- **Task 1:** Ek dummy .csv file banayein jisme aapke top 5 favorite machine learning models aur unke pros/cons hon. Use CSVLoader to load it and print the page_content of the 3rd row.
- **Task 2:** Write a script that uses .lazy_load() with a CSV file and only prints the first 5 rows to save memory, then break the loop.

Ab jab humne documents load kar liye hain, ek problem aati hai: agar PDF 500 pages ki hai, toh

wo LLM ki token limit ko break kar degi. Isko handle karne ke liye kya hum next class mein **Text Splitters and Chunking strategies** (jo lambe text ko smart pieces mein todte hain) cover karein?